

CS/ECE 374 A ✧ Fall 2026

🌀 Homework 5 🌀

Due Tuesday, October 6, 2026 at 9pm Central Time

Please review and understand the homework policies, especially [our expectations about describing algorithms](#), and both the [general and more specific grading rubrics](#) for different types of algorithms.

You may use any algorithm or data structure seen in class as a black box without having to explain how it works.

Both problems below involve arrays of numbers. Your algorithms must not use hashing, radix sort, or any other method that depends upon the precise numbers given, as opposed to their order.

1. Let $I_1, I_2, \dots, I_n$ be a collection of n non-empty real intervals. Each interval I_i is specified by its two endpoints a_i and b_i with $a_i < b_i$. The **overlap length** $overlap(i, j)$ between two intervals I_i and I_j is the length of the longest subinterval shared between I_i and I_j if they overlap or 0 if I_i and I_j do not overlap. Precisely,

$$overlap(i, j) := \max \{0, \min \{b_i, b_j\} - \max \{a_i, a_j\}\}.$$

Describe an algorithm to compute the maximum overlap length between any pair of *distinct* intervals I_i and I_j . You may assume the intervals are given by two arrays $A[1..n]$ and $B[1..n]$ where $A[i] = a_i$ and $B[i] = b_i$ for all $i \in \{1, \dots, n\}$. You may assume all array values are distinct.

For example, given $A = \langle 1, 7, 4, 5 \rangle$ and $B = \langle 8, 10, 9, 6 \rangle$, your algorithm should return 4 for the intervals $I_1 = [1, 8]$ and $I_3 = [4, 9]$.

[Hint: A $O(n^2)$ time algorithm is straightforward to find, but it doesn't use anything from class. Can you do better?]

[Hint: Suppose you knew a value x such that $a_i < x$ and $a_j \geq x$ where I_i and I_j are the maximal overlapping pair of intervals. How would you find I_i and I_j ? Can you design a divide-and-conquer algorithm that uses this idea?]

2. Suppose we are given a set S of n items, each with a *value* and a *weight*. The values and weights are positive and distinct. For any element $x \in S$, we define two subsets:

- $S_{<x}$ is the set of elements of S whose value is less than the value of x .
- $S_{>x}$ is the set of elements of S whose value is more than the value of x .

For any subset $R \subseteq S$, let $w(R)$ denote the sum of weights of elements in R . The **weighted median** of R is the element x such that $w(S_{<x}) \leq w(S)/2$ and $w(S_{>x}) < w(S)/2$. In other words, if one were to sort the items in S by *value* and then start summing the weights from the beginning of the sorted list, the weighted median is the first element where the current sum passes half the weight of the entire set.

Describe an algorithm to compute the weighted median of a given weighted set. You may assume your input consists of two *unsorted* arrays $S[1..n]$ and $W[1..n]$ where for each index i , the i th element of the set has value $S[i]$ and weight $W[i]$.

[Hint: This problem is straightforward to solve in $O(n \log n)$ time by sorting, but that strategy barely uses anything new to this class. Can you do better?]

*3. Practice only. Do not submit solutions.

The following variant of the infamous StoogeSort algorithm¹ was discovered by the British actor Patrick Troughton during rehearsals for the 20th anniversary *Doctor Who* special “The Five Doctors”.²

```

WHOSORT( $A[1..n]$ ):
  if  $n < 13$ 
    sort  $A$  by brute force
  else
     $k = \lceil n/5 \rceil$ 
    WHOSORT( $A[1..3k]$ )      ⟨⟨Hartnell⟩⟩
    WHOSORT( $A[2k+1..n]$ )    ⟨⟨Troughton⟩⟩
    WHOSORT( $A[1..3k]$ )      ⟨⟨Pertwee⟩⟩
    WHOSORT( $A[k+1..4k]$ )    ⟨⟨Davison⟩⟩

```

- Prove by induction that WHOSORT correctly sorts its input. [Hint: Where can the smallest k elements be?]
- Would WHOSORT still sort correctly if we replaced “if $n < 13$ ” with “if $n < 4$ ”? Justify your answer.
- Would WHOSORT still sort correctly if we replaced “ $k = \lceil n/5 \rceil$ ” with “ $k = \lfloor n/5 \rfloor$ ”? Justify your answer.
- What is the running time of WHOSORT? (Set up a running-time recurrence and then solve it, ignoring the floors and ceilings.)
- Forty years later, 15th Doctor Ncuti Gatwa discovered the following optimization to WHOSORT, which uses the standard MERGE subroutine from mergesort, which merges two sorted arrays into one sorted array.

```

NuWHOSORT( $A[1..n]$ ):
  if  $n < 13$ 
    sort  $A$  by brute force
  else
     $k = \lceil n/5 \rceil$ 
    NuWHOSORT( $A[1..3k]$ )      ⟨⟨Grant⟩⟩
    NuWHOSORT( $A[2k+1..n]$ )    ⟨⟨Whittaker⟩⟩
    MERGE( $A[1..2k]$ ,  $A[2k+1..4k]$ ) ⟨⟨Tennant⟩⟩

```

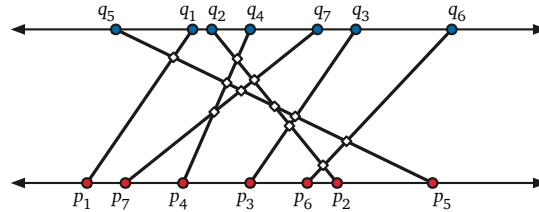
What is the running time of NuWHOSORT?

¹https://en.wikipedia.org/wiki/Stooge_sort

²Tom Baker, the fourth Doctor, declined to return for the reunion; hence, only four Doctors appeared in “The Five Doctors”. (Well, okay, technically the BBC used excerpts of the unfinished episode “Shada” to include Baker, but he wasn’t really *there*—to the extent that any fictional character in a television show about a time traveling wizard arguing with several other versions of himself about immortality can be said to be “really” “there”.)

Solved problems

4. Suppose we are given two sets of n points, one set $\{p_1, p_2, \dots, p_n\}$ on the line $y = 0$ and the other set $\{q_1, q_2, \dots, q_n\}$ on the line $y = 1$. Consider the n line segments connecting each point p_i to the corresponding point q_i . Describe and analyze a divide-and-conquer algorithm to determine how many pairs of these line segments intersect, in $O(n \log n)$ time. See the example below.



Seven segments with endpoints on parallel lines, with 11 intersecting pairs.

Your input consists of two arrays $P[1..n]$ and $Q[1..n]$ of x -coordinates; you may assume that all $2n$ of these numbers are distinct. No proof of correctness is necessary, but you should justify the running time.

Solution: We begin by sorting the array $P[1..n]$ and permuting the array $Q[1..n]$ to maintain correspondence between endpoints, in $O(n \log n)$ time. Then for any indices $i < j$, segments i and j intersect if and only if $Q[i] > Q[j]$. Thus, our goal is to compute the number of pairs of indices $i < j$ such that $Q[i] > Q[j]$. Such a pair is called an ***inversion***.

We count the number of inversions in Q using the following extension of mergesort; as a side effect, this algorithm also sorts Q . If $n < 100$, we use brute force in $O(1)$ time. Otherwise:

- Color the elements in the Left half $Q[1.. \lfloor n/2 \rfloor]$ **blue**.
- Color the elements in the Right half $Q[\lfloor n/2 \rfloor + 1..n]$ **red**.
- Recursively count inversions in (and sort) the **blue** subarray $Q[1.. \lfloor n/2 \rfloor]$.
- Recursively count inversions in (and sort) the **red** subarray $Q[\lfloor n/2 \rfloor + 1..n]$.
- Count **red/blue** inversions as follows:
 - MERGE the sorted subarrays $Q[1..n/2]$ and $Q[n/2+1..n]$, maintaining the element colors.
 - For each **blue** element $Q[i]$ of the now-sorted array $Q[1..n]$, count the number of smaller **red** elements $Q[j]$.

The last substep can be performed in $O(n)$ time using a simple for-loop:

```

COUNTREDBLUE(A[1..n]):
    count ← 0
    total ← 0
    for i ← 1 to n
        if A[i] is red
            count ← count + 1
        else
            total ← total + count
    return total

```

MERGE and COUNTREDBLUE each run in $O(n)$ time. Thus, the running time of our inversion-counting algorithm obeys the mergesort recurrence $T(n) = 2T(n/2) + O(n)$. (We can safely ignore the floors and ceilings in the recursive arguments.) We conclude that the overall running time of our algorithm is $O(n \log n)$, as required.

Rubric: This is enough for full credit.

In fact, we can execute the third merge-and-count step directly by modifying the MERGE algorithm, without any need for “colors”. Here changes to the standard MERGE algorithm are indicated in red.

```

MERGEANDCOUNT(A[1..n], m):
    i ← 1; j ← m + 1; count ← 0; total ← 0
    for k ← 1 to n
        if j > n
            B[k] ← A[i]; i ← i + 1; total ← total + count
        else if i > m
            B[k] ← A[j]; j ← j + 1; count ← count + 1
        else if A[i] < A[j]
            B[k] ← A[i]; i ← i + 1; total ← total + count
        else
            B[k] ← A[j]; j ← j + 1; count ← count + 1
    for k ← 1 to n
        A[k] ← B[k]
    return total

```

We can further optimize MERGEANDCOUNT by observing that *count* is always equal to $j - m - 1$, so we don’t need an additional variable. (Proof: Initially, $j = m + 1$ and $count = 0$, and we always increment j and $count$ together.)

```

MERGEANDCOUNT2( $A[1..n], m$ ):
   $i \leftarrow 1$ ;  $j \leftarrow m + 1$ ;  $total \leftarrow 0$ 
  for  $k \leftarrow 1$  to  $n$ 
    if  $j > n$ 
       $B[k] \leftarrow A[i]$ ;  $i \leftarrow i + 1$ ;  $total \leftarrow total + j - m - 1$ 
    else if  $i > m$ 
       $B[k] \leftarrow A[j]$ ;  $j \leftarrow j + 1$ 
    else if  $A[i] < A[j]$ 
       $B[k] \leftarrow A[i]$ ;  $i \leftarrow i + 1$ ;  $total \leftarrow total + j - m - 1$ 
    else
       $B[k] \leftarrow A[j]$ ;  $j \leftarrow j + 1$ 
  for  $k \leftarrow 1$  to  $n$ 
     $A[k] \leftarrow B[k]$ 
  return  $total$ 

```

MERGEANDCOUNT2 still runs in $O(n)$ time, so the overall running time is still $O(n \log n)$, as required. ■

Rubric: 10 points = 2 for base case + 2 for divide (split and recurse) + 4 for conquer (merge and count) + 2 for time analysis. This is neither the only way to correctly describe this algorithm nor the only correct $O(n \log n)$ -time algorithm. No proof of correctness is required.

Max 3 points for a correct $O(n^2)$ -time algorithm.

Notice that each boxed algorithm is preceded by a clear English description of the task that algorithm performs—not how the algorithm works, but the relationship between its input and its output. **Each English description is worth 25% of the credit for that algorithm** (rounding to the nearest half-point). For example, the COUNTREDBLUE algorithm is worth 4 points (“conquer”); the English description alone (“For each blue element $Q[i]$ of the now-sorted array $Q[1..n]$, count the number of smaller red elements $Q[j]$.”) is worth 1 point.